

LeetCode 114 - Flatten Binary Tree to Linked List

This is one of the most frequently asked tree interview questions.

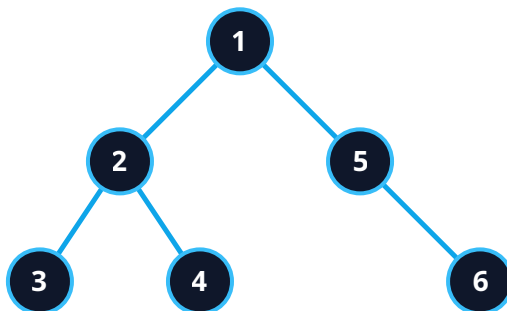
The goal is to convert a binary tree into a linked list in-place.

The linked list should follow the Preorder Traversal.

Preorder is

- Root
- Left
- Right

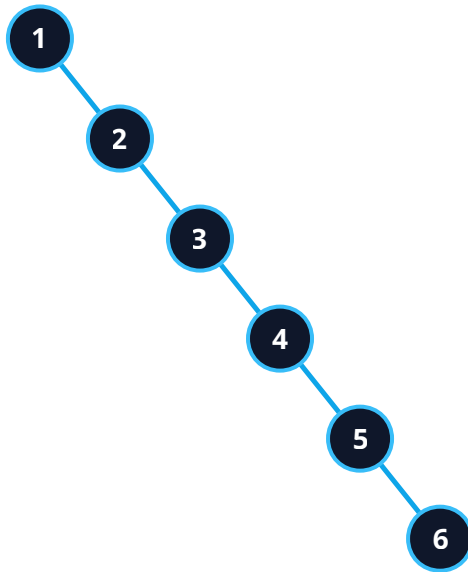
For example,



Preorder is

1 2 3 4 5 6

So after flattening, the tree becomes



Every

```
left = NULL
```

and

```
right = next preorder node
```

Three Approaches

Approach	Time	Space
Reverse Preorder Recursion	$O(N)$	$O(H)$
Stack (Iterative DFS)	$O(N)$	$O(H)$
Morris Traversal	$O(N)$	$O(1)$

Now let's understand each one deeply.

APPROACH 1 (Reverse Preorder Recursion)

```
flatten(right)
flatten(left)
```

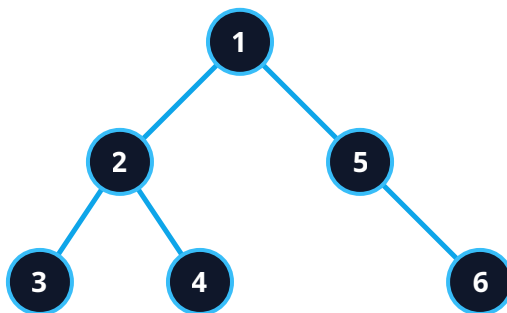
Most students ask

Why Right first?

This is the entire trick.

First Understand Normal Preorder

Suppose tree is



Preorder

1
2
3
4
5
6

Suppose we try recursion.

Problem

When we finish processing left subtree,
we don't yet know where its last node is.

Example

1
↓
2
↓
3

How do we connect

3 → 4

or

4 → 5

We don't know.

Reverse Thinking

Instead of building

Front → Back

Build

Back → Front

Exactly like reversing a linked list.

Desired order

1
2
3
4
5
6

Reverse order

6
5
4
3
2
1

How do we visit in this order?

Reverse preorder

Right



Left



Root

That is exactly

```
flatten(root->right);  
flatten(root->left);
```

Role of prev

Initially

```
prev = NULL
```

prev always means

Head of the already flattened list.

Initially

```
NULL
```

After processing 6

```
6 -> NULL
```

prev

```
6
```

After processing 5

```
5 -> 6
```

prev

```
5
```

After processing 4

```
4 -> 5 -> 6
```

prev

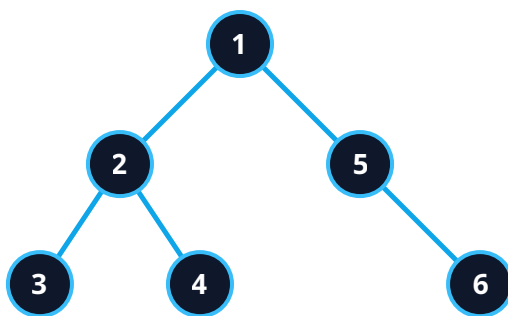
```
4
```

Eventually

```
1→2→3→4→5→6
```

Complete Dry Run

Tree



Call

```
flatten(1)
```

Go Right

```
flatten(5)
```

Go Right

```
flatten(6)
```

Go Right

```
NULL
```

Return

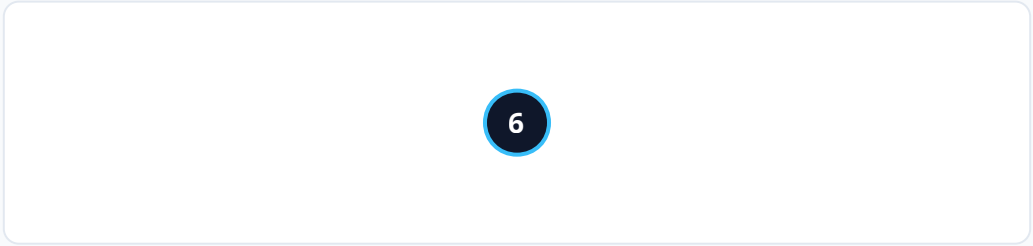
Go Left

```
NULL
```

Return

Now process 6

```
6->right=prev(NULL)
```



A diagram of a linked list node. It consists of a light yellow rounded rectangle. In the center of the rectangle is a dark blue circle with a light blue border, containing the number 6 in white.

prev=6

Current list

```
6
```

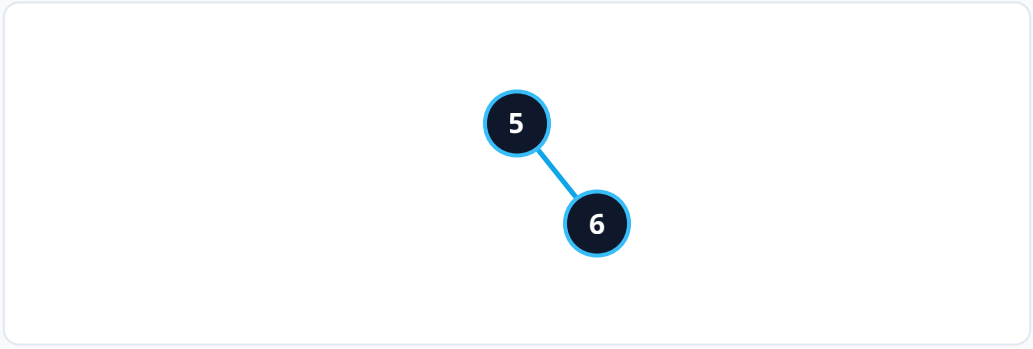
Return to 5

Left

```
NULL
```

Now process

```
5->right=6
```



prev

5

Current list

5→6

Return to 1

Now Left

flatten(2)

Go Right

flatten(4)

Right NULL

Left NULL

Process

4→5→6

prev

4

Return

Go Left

flatten(3)

Process

3→4→5→6

prev

3

Return to 2

Process

2→3→4→5→6

prev

2

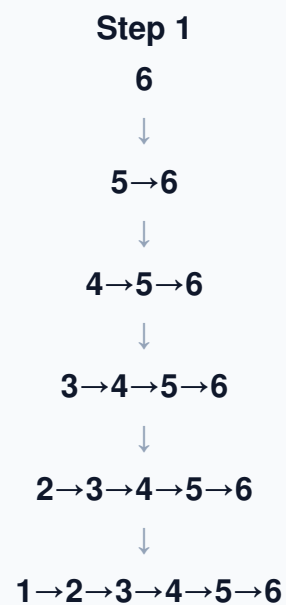
Return to 1

Process

1→2→3→4→5→6

Done.

Visualization



Notice

The list is growing backwards.

Complexity

Time

Every node once
 $O(N)$

Space

Recursive stack
 $O(H)$

APPROACH 2 (Stack)

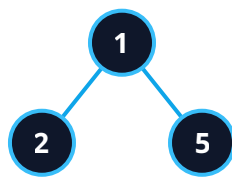
This exactly simulates preorder traversal.
Instead of recursion,
we use stack.

Why Push Right First?

Stack is

LIFO

Suppose



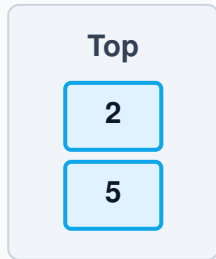
We want preorder

1
↓
2
↓
5

Stack pops last inserted.
Therefore push

Right
then
Left

Stack



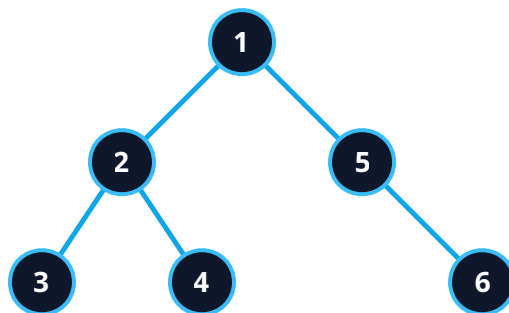
Next pop

2

Perfect.

Dry Run

Initial tree



Stack



Pop

1

Push

5
2

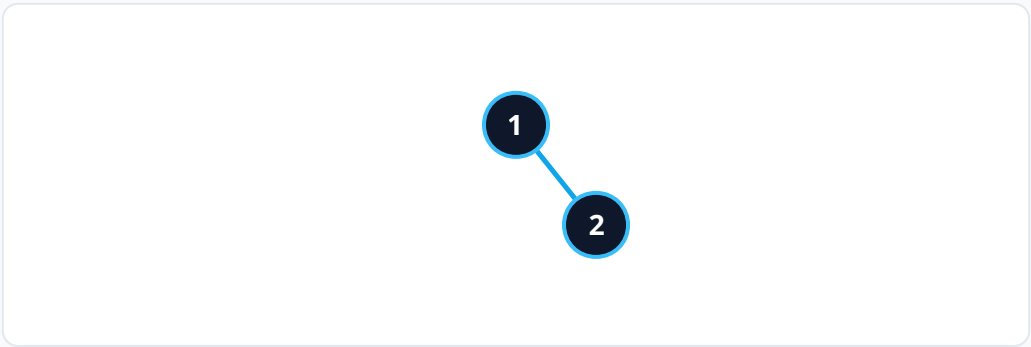
Stack



Link

1→2

Current



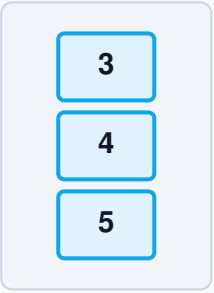
Pop

2

Push

4
3

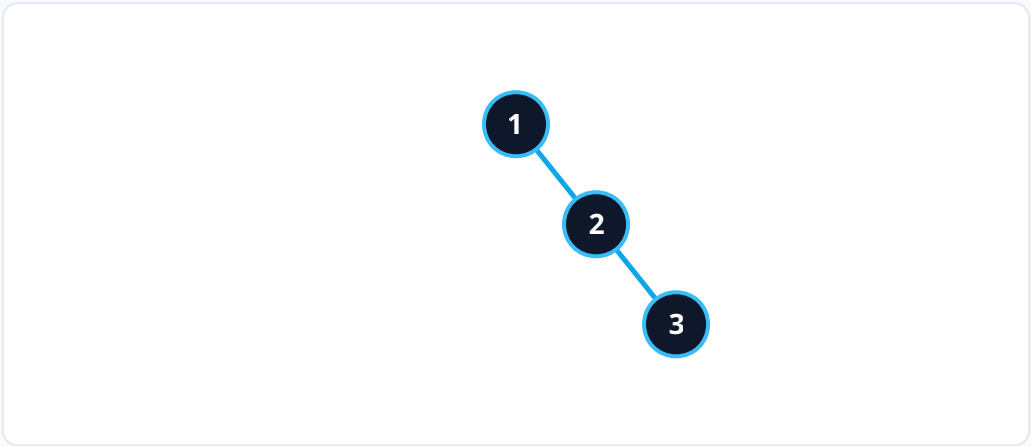
Stack



Link



Tree



Pop



No children

Top



Link



Pop



Top

5

Link

4→5

Pop

5

Push

6

Link

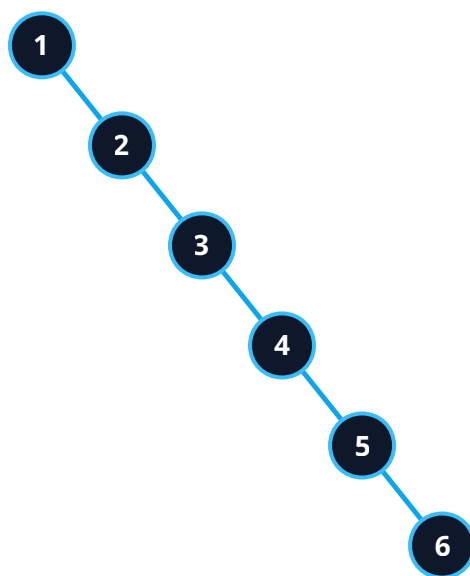
5→6

Pop

6

Done.

Final



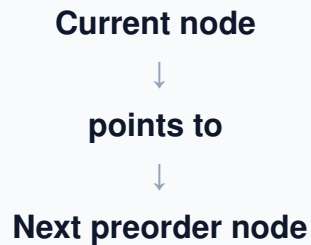
Why This Works

The stack always contains the next preorder node that has not yet been processed.

So

```
cur->right = st.top();
```

means



Exactly what we want.

Complexity

Time

```
O(N)
```

Space

```
O(H)
```

Worst

```
O(N)
```

APPROACH 3 (Morris Traversal)

This is the interview favorite.

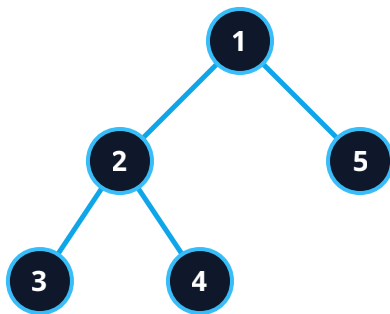
Space

```
O(1)
```

- No recursion
- No stack

Main Idea

Suppose



Current

1

Its preorder is

1
↓
2
↓
3
↓
4
↓
5

Notice

Entire left subtree comes before right subtree.

So we need

1
↓
Left subtree
↓
Right subtree

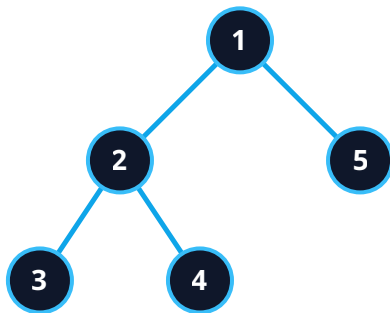
Question

How do we attach the right subtree after the entire left subtree?

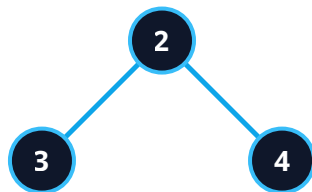
Answer

Find the rightmost node of the left subtree.

For current



Left subtree



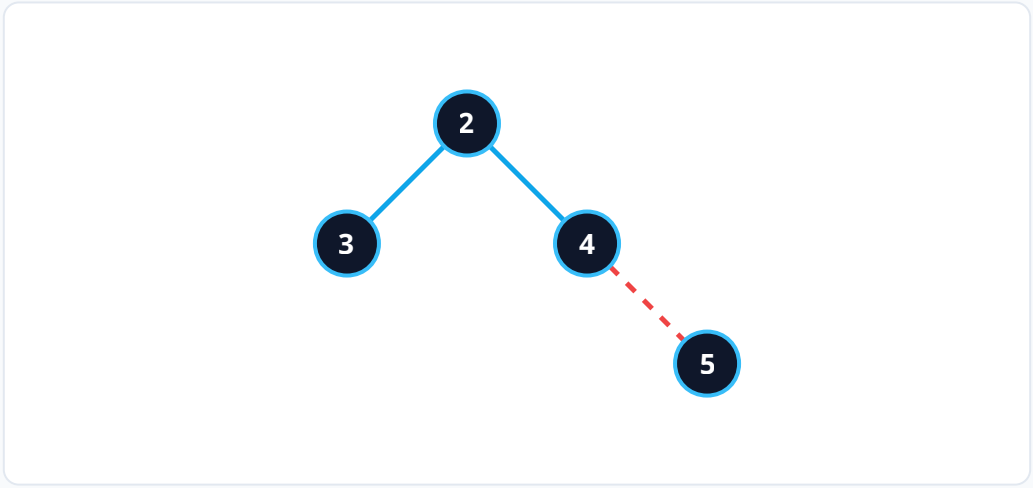
Rightmost node

4

Attach

4→5

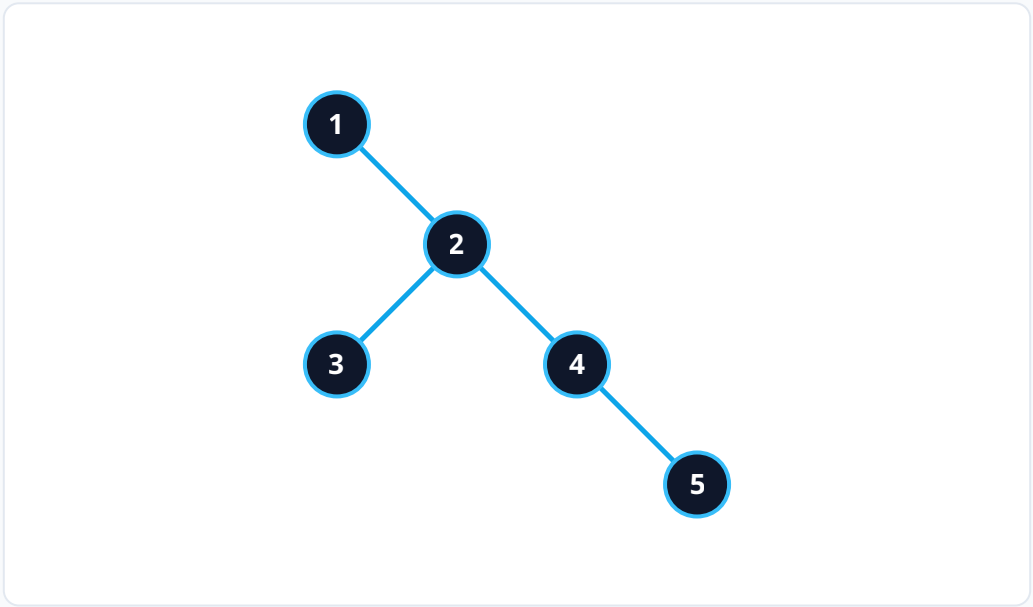
Now



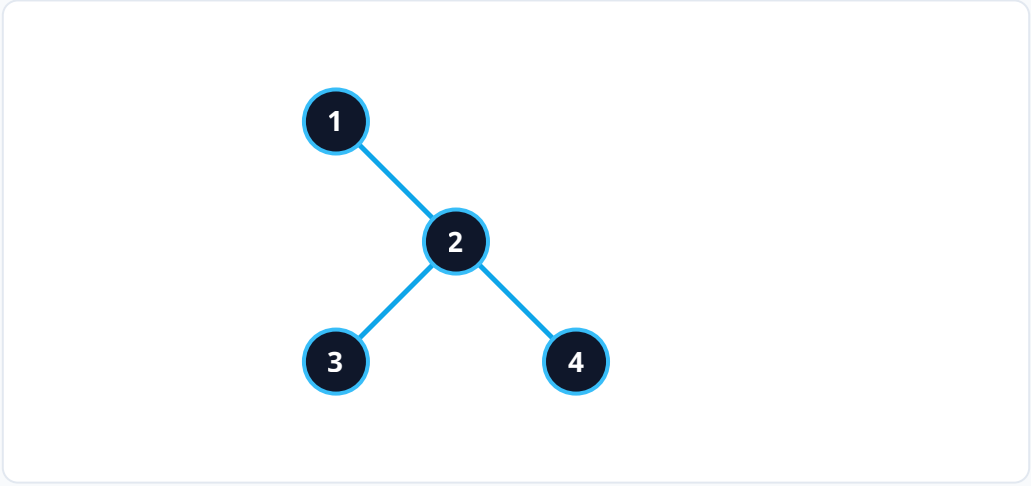
Now move left subtree



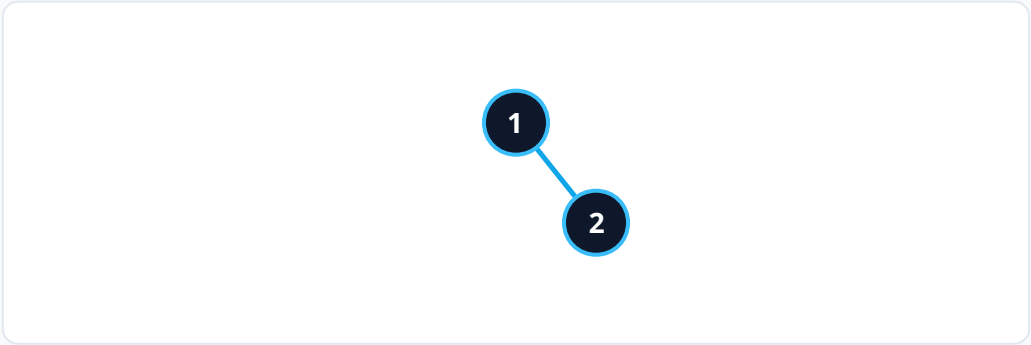
Result



Remove left



becomes

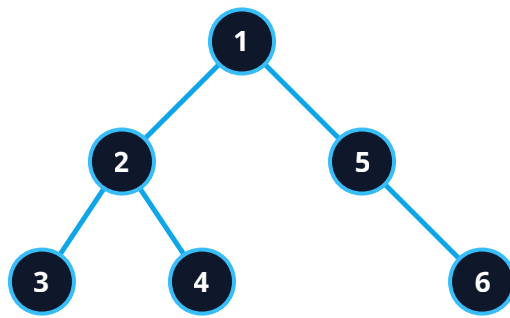


Continue.
Eventually



Complete Dry Run

Initial



Current

1

Left exists.

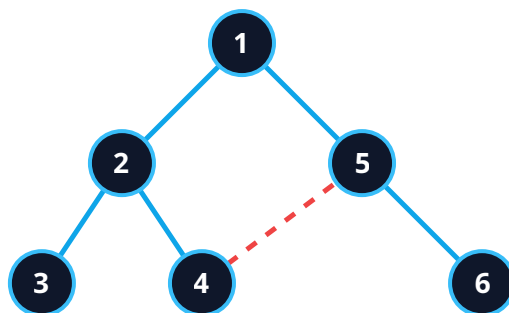
Find predecessor.

4

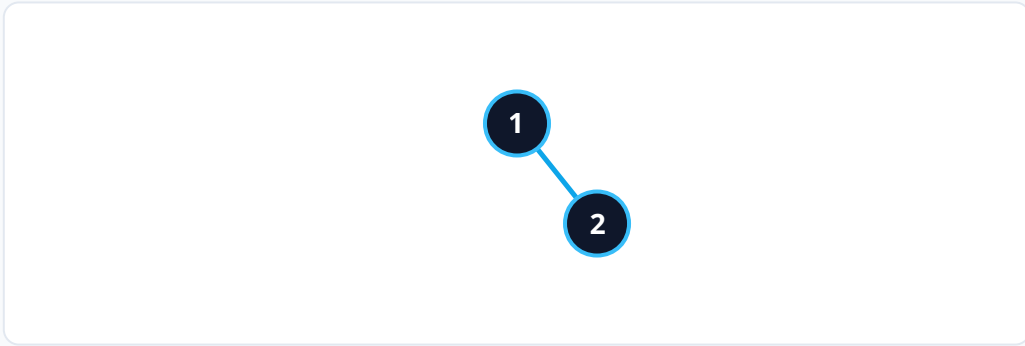
Connect

4→5

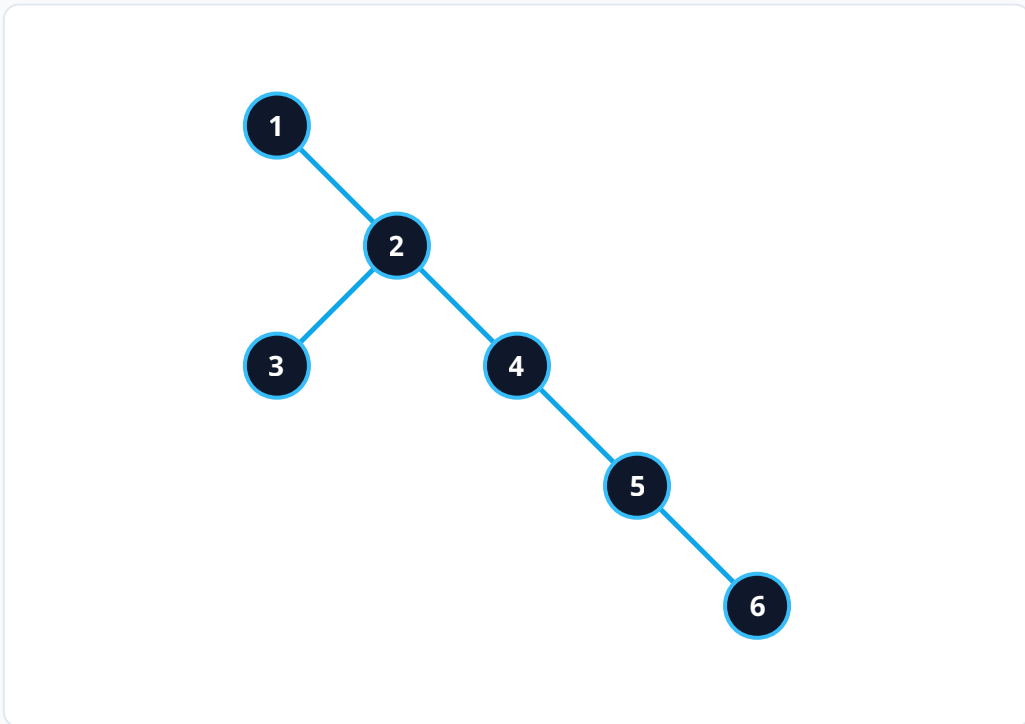
Now



Move left



Remove left.
Tree



Current

2

Find predecessor

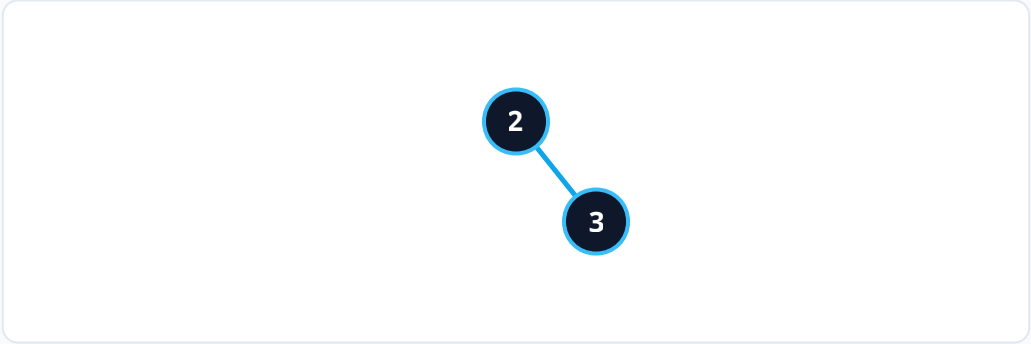
3

Actually the rightmost node in the left subtree of 2 is 3 (because 3 has no right child).

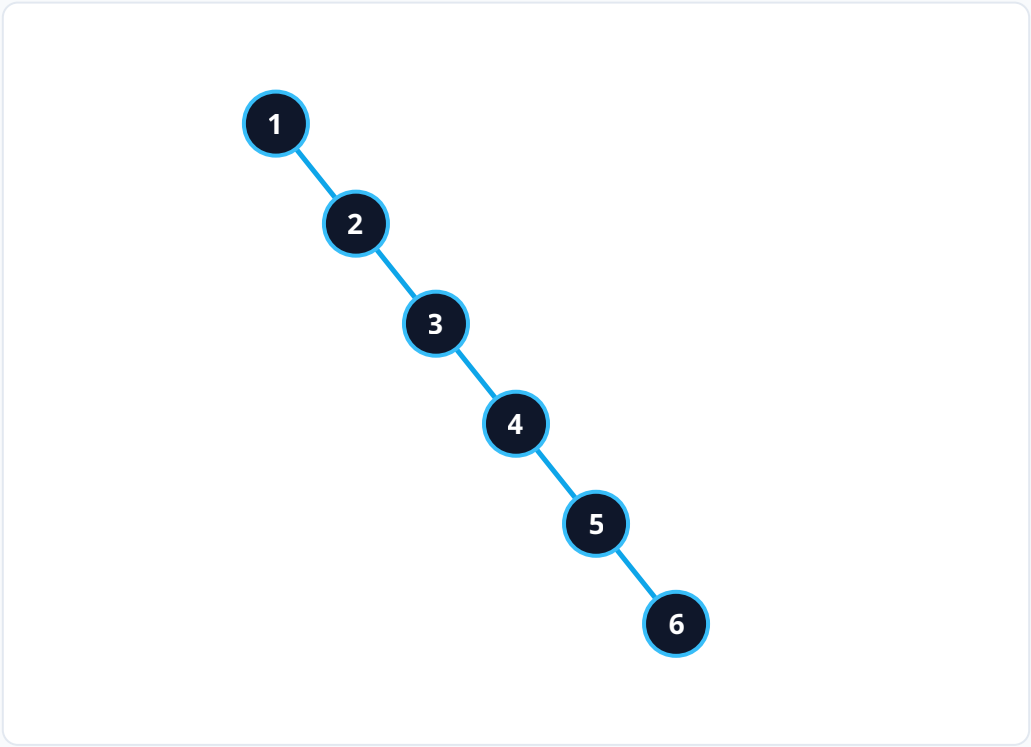
Connect

3→4

Move left



Tree



Current

3

No left
Move

4

Current

4

No left
Move

5

Current

5

No left

Move

6

Current

6

No left

Move

Done .

Why Is It Called Morris Traversal?

Morris Traversal temporarily rewires the tree by creating links so that we can traverse it without recursion or an explicit stack. In this flattening algorithm, we use the same idea of rewiring pointers, but instead of restoring the original tree, we permanently transform it into the required linked list.

Comparison of the Three Approaches

Feature	Reverse Preorder	Stack	Morris
Traversal order	Reverse Preorder (Right → Left → Root)	Preorder (Root → Left → Right)	Preorder (by rewiring)
Extra Space	O(H)	O(H)	O(1)
Recursion	Yes	No	No
Stack Used	Implicit (call stack)	Explicit stack	None
In-place	Yes	Yes	Yes
Interview Difficulty	Medium	Easy	Hard

Which One Should You Use?

- **Learning recursion and tree backtracking:** Reverse preorder is the cleanest and easiest to reason about.
- **If recursion depth may be a problem:** Use the iterative stack approach.
- **For the best space complexity and advanced interviews:** Use the Morris traversal approach. It achieves O(N) time with O(1) extra space while modifying the tree in place.